The engine behind agentic detection engineering.

Spec-driven development for detection engineering: an ontology and a taxonomy, an engine you pin, and threat-aligned detections that ship fast.



opentide, in one sentence

opentide is spec-driven development for detection engineering: an ontology and a taxonomy that keep every detection threat-aligned, maintainable and fast to ship, with or without agents.

Strict validation, a deployment pipeline, modern DevOps workflows and agent-native constructs, on an object graph that scales with you.

01 **Why it matters**
the gap · five failure modes · new economics

02 **What opentide is**
spec · object graph · engine · platforms

03 **Why we re-released it**
from submodule to CLI, built for agents

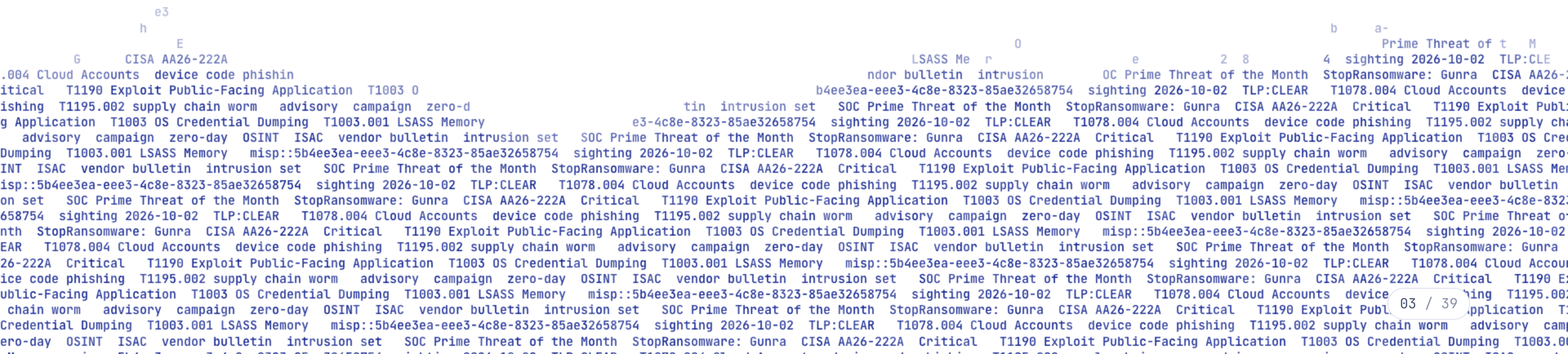
04 **Agentic detection engineering**
the loop · a worked example · guardrails

05 **What comes next**
velocity · roadmap · north star

01 / 05

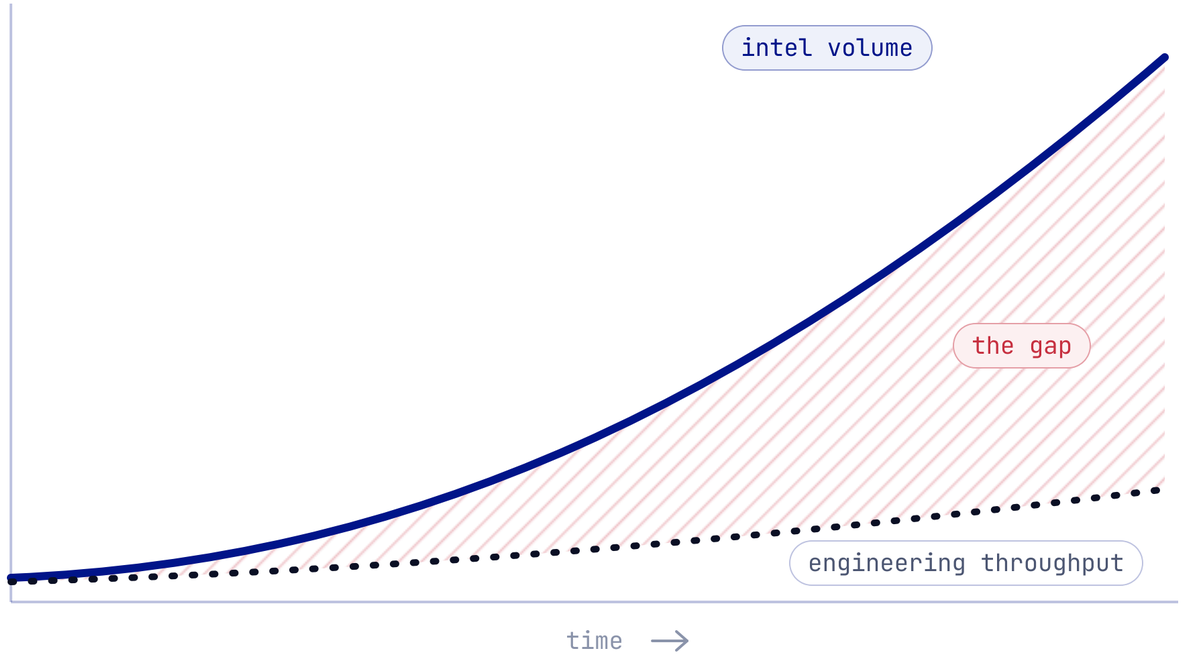
Why it matters

Intelligence moves faster than detection engineering can. And agents just changed what is scarce.



Intelligence outruns detection engineering.

The bottleneck was never the strategy. It was the capacity to execute it: read, model, write, tune, deploy, repeat.



1,000s

intel items a month: advisories, CVEs, campaigns

hours

per TTP to read, model, write, tune and deploy

tens

production-ready detection rules a month

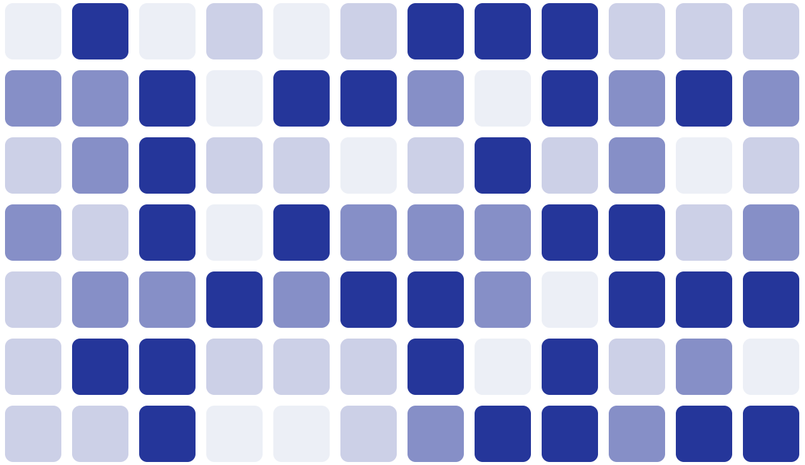
Five ways detection engineering breaks at scale.

01	**Content drift**	Two engineers describe the same rule two ways. Reviews catch style, not correctness.
02	**No thread from threat to rule**	A rule exists, but why? Which threat, which objective, what breaks if it is deleted?
03	**Multi-platform sprawl**	One idea, re-implemented per SIEM and EDR. Cross-platform coverage is guesswork.
04	**Copy-paste reuse**	Submodules and forks couple repositories. Upgrades spread breaking changes silently.
05	**Unsafe automation**	Scripts and agents edit YAML with no schema and no quality gate.

Coverage is a graph, not a heatmap.

A technique heatmap flatters. A threat-informed graph says which threat, which signal, which rule, on which platform.

“We cover 85% of ATT&CK.”



for which threats? with which signals?
on which platforms? deployed where?

dimension	att&ck heatmap	opentide graph
Granularity	technique	**atomic threat vector**
Context	generic	**surface and terrain**
Impact	implicit	**severity · impact · leverage · viability**
Chaining	none	**threat → threat attack paths**
Intel link	manual	**actors, sightings, references**
Detection	separate	**objective → signals → rules**

Agents made the query cheap. Not threat-aligned.

Writing a detection query



Aligning and maintaining it



Threat alignment, coverage and maintenance did not get cheaper with the text. A spec has to carry that work.

what becomes scarce

A spec

An ontology and a taxonomy for what a detection is.

Threat alignment

Every rule traces to an objective and a threat.

Quality gates

Strict validation in CI, for humans and agents alike.

Maintainability

One model across seven platforms, versioned in git.

That is what opentide is.

next: what opentide is →

02 / 05

What opentide is

Spec-driven development for detection engineering: an ontology, a taxonomy, and an engine you pin.



Spec-driven development for detection engineering.

the spec · ontology + taxonomy

specifications

normative · CC-BY-4.0

Ontology: `threat::1.0` · `objective::1.0` · `rule::1.1`

Taxonomy: 36 versioned vocabularies · fixtures · RFCs

`opentidehq/specifications`

↓ spec markdown → Pydantic models → JSON Schema (generated, never edited)

the engine you pin

opentide

pip install opentide · EUPL-1.2

One package: CLI, Python SDK, MCP server, 7 platform adapters

Types and JSON Schema share one model, so they cannot drift

`opentidehq/opentide`

↓ validates · generates · deploys · documents

the source of truth

your detection repo

yours · in git

`objects/` threats, objectives and rules as YAML

.opentide/configurations · your CI · your reviews · your history

`private or public`

The spec does not ship detections. The package does not hold your YAML. **Your repo stays the source of truth.**

Three objects. Three questions.

TVM · threat

threat::1.0

Threat

What do we defend against?

severity · impact · leverage · viability · terrain · surface · mitre_attack · actors · chaining

DOM · objective

objective::1.0

Objective

What are we trying to detect?

priority · type · composition · threats[ ] · signals[ ]: methodology, entities, data

MDR · rule

rule::1.1

Rule

How do we detect it, on which platform?

status · severity · techniques · detection_model · configurations. {sentinel, splunk, …}



References point back

rule.detection_model → objective · objective.threats → threat



Coverage flows forward

threats covered by objectives, implemented by rules

One chain. Three files. Linked by UUID.

objects/threats/gunra-ransomware-intrusion.yaml

```yaml
name: Gunra ransomware intrusion
criticality: High
metadata:
  uuid: aa36f6de-4cc8-…
  schema: threat::1.0
  tlp: clear
threat:
  severity: Significant incident
  impact: [Data Breach]
  leverage: [Infrastructure Compromise]
  viability: Likely
  surface:
    - Windows::Server
  mitre_attack: [T1190, T1003]
  killchain: Exploitation
```

objects/objectives/credential-access.yaml

```yaml
name: Credential access via LSASS
metadata:
  uuid: f8817f95-c43e-…
  schema: objective::1.0
objective:
  priority: High
  type: Threat
  composition:
    strategy: synergetic
  threats:
    - aa36f6de-4cc8-…
  signals:
    - name: LSASS process access
      methodology: Behavioural
      entities: [Process, Hostname]
      data:
        availability: Complete
```

objects/rules/lsass-memory-access.yaml

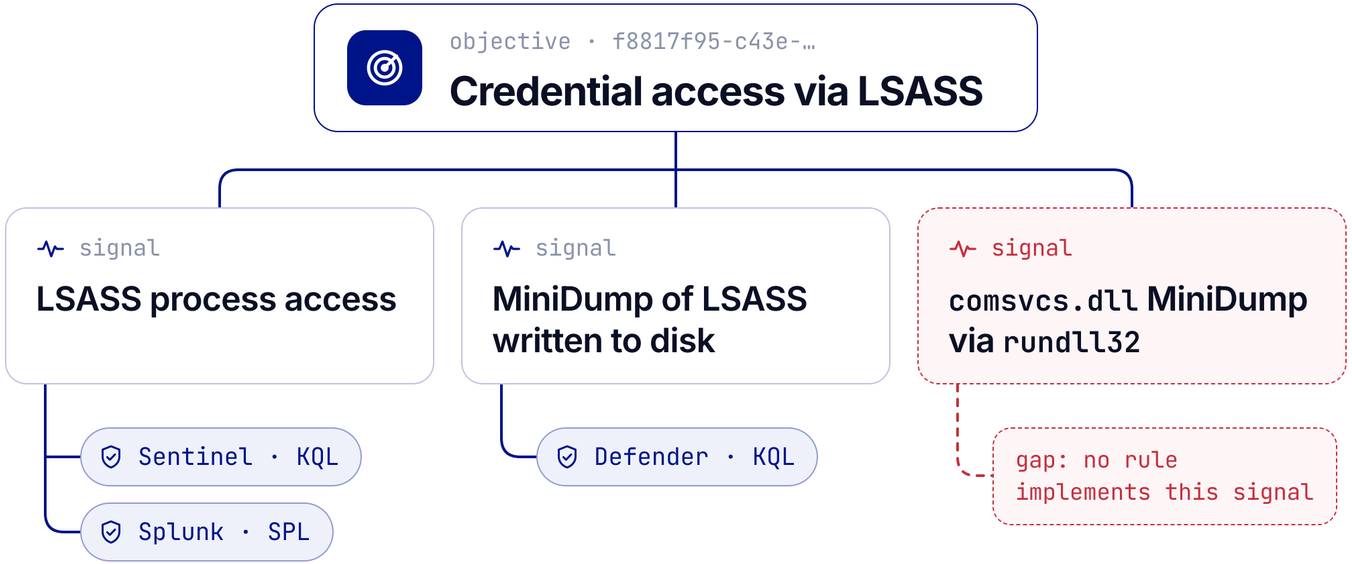
```yaml
name: LSASS memory access
metadata:
  uuid: c8f282eb-0286-…
  schema: rule::1.1
  reviewed: "2026-10-01"
status: STAGING
severity: High
techniques: [T1003.001]
detection_model: f8817f95-c43e-…
configurations:
  sentinel:
    query: |
      DeviceProcessEvents
      | where FileName =~ "lsass.exe"
  splunk:
    query: |
      `sysmon` EventCode=10
```

aa36… objective.threats → the threat f881… rule.detection_model → the objective

excerpts · rename or move files freely, links hold

The signal is the unit of coverage.

Objectives decompose into signals, rules implement them, and coverage is a walk of the graph. Gaps are visible, not guessed.



`rule.detection_model` → an objective uuid or a signal uuid · since 0.10

MCP · coverage — tool call

```
coverage(technique="T1003")
```

```
{
  "technique": "T1003",
  "covered": true,
  "matched_techniques": ["T1003.001"],
  "rules": ["c8f282eb-0286-…",
            "6a1f3f4b-ac54-…"]
}
```

the same question, from the CLI

```
$ opentide info coverage --technique T1003
```

what the graph makes visible

validation errors

- × orphan rule
- × dangling reference
- × duplicate UUID
- × circular chain

valid, but a visible gap

- coverage island

One pipeline: author, generate, validate, deploy, document.

The commands you run on a laptop are the commands CI runs.

01

Author

Threats, objectives and rules as YAML, from generated templates.

`objects/*.yaml`

02

Generate

Deterministic schemas, templates, vocabularies, snippets. Safe to commit.

`opentide generate`

03

Validate

Schema, unique UUIDs, references, vocabularies. Exit 1 blocks the merge.

`opentide validate --strict`

04

Deploy

Per platform and tenant. Staging first, promotion behind approval.

`opentide deploy`

05

Document

Wiki pages with chaining diagrams and ATT&CK coverage.

`opentide generate docs`

ci, generated by `opentide setup ci github|gitlab|azure`

pins the opentide version you run

pull request: `validate --strict` → `lint --strict` → `validate query --live` → `generate` → `deploy --plan STAGING`

default branch: protected environment · separate approval → `deploy --plan PRODUCTION`

One model. Seven platforms. No rewrites.

platform	--platform	query language	deploy	tenant query check
Microsoft Sentinel	`sentinel`	KQL	✓	✓ `--live`
Defender for Endpoint	`defender_for_endpoint`	KQL	✓	✓ `--live`
Splunk Enterprise Security	`splunk`	SPL	✓	✓ `--live`
SentinelOne	`sentinel_one`	S1QL	✓	✓ `--live`
Carbon Black Cloud	`carbon_black_cloud`	Lucene	✓	✓ `--live`
CrowdStrike Falcon	`crowdstrike`	—	✓	✗ `supported: false`
HarfangLab	`harfanglab`	—	✓	✗ `supported: false`

Five platforms get a live query check in CI. CrowdStrike and HarfangLab are **deploy-only today**, and flagged as such.

```
validate query --platform crowdstrike                                --json

{ "platform": "crowdstrike", "valid": null, "supported": false,
  "ok": false, "status": "failed",
  "message": "query validation not supported for crowdstrike" }
```

next `Elastic Security · RFC 0007, accepted` `Sysdig Secure + Falco · RFC 0004, draft`

Three doors, one engine.

The CLI, the Python SDK and the MCP server share one object model, one validator, one deployer.

CLI

humans and CI pipelines

terminal

```
$ opentide validate \
    --uuid c8f282eb-… --strict
# exit 0 success · 1 error · 2 usage
```

SDK

Python tools and tests

python

```python
from opentide import Opentide
Opentide.initialise()
rule = Opentide.Rules["c8f282eb-…"]
result = rule.validate()
print(result.ok, result.errors)
```

MCP

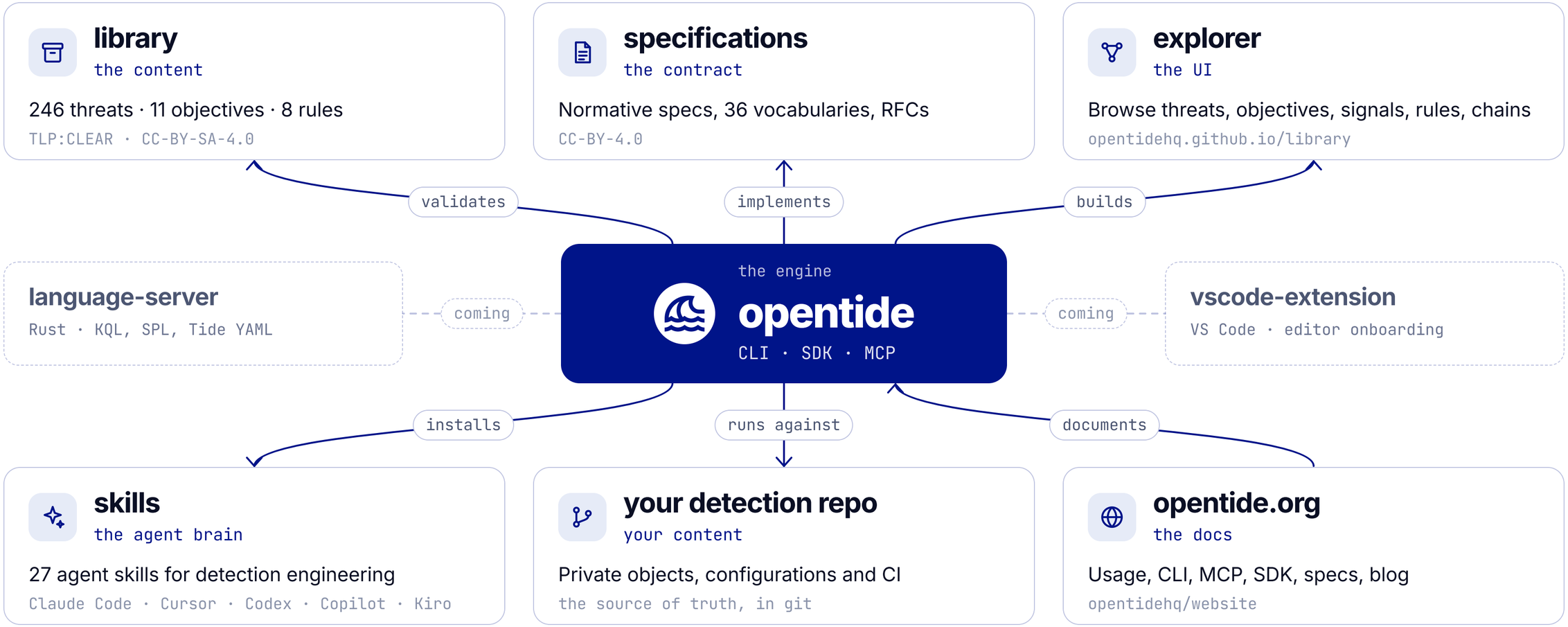
AI agents in the editor

tool call

```
validation_report(uuid="c8f282eb-…")
# returns a structured issue list
```



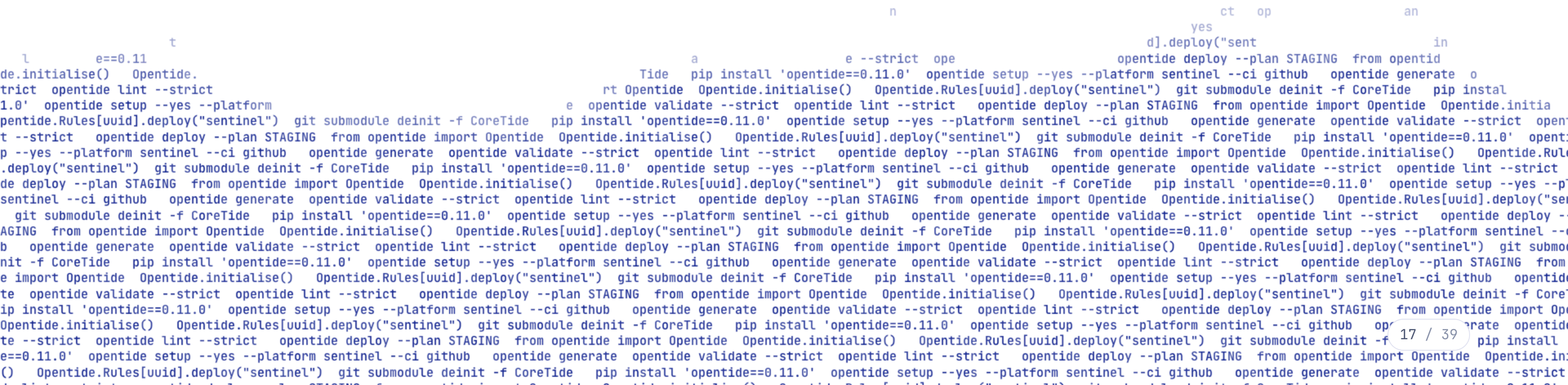
Each repository has one job.



03 / 05

Why we re-released it

Years in production as a git submodule. Now a versioned CLI on PyPI, with the ergonomics humans and agents both need.



Battle-hardened before it was a package.

years in production · CoreTide

2023 · Hack.lu

TIDeMEC, homegrown at the EC

A detection engineering platform built inside the European Commission CSOC.

2024 · FIRST TC Amsterdam

The first opentide project

CoreTIDE presented as the backbone of a family of tools.

Feb 2025 · open source

CoreTide on GitHub, EUPL-1.2

Published after more than two years in production.

Mar 2026 · FIRST CTI

AgenTide

6 agent pull requests, 30+ objects, about 15 minutes each.

the re-release · opentide

13 Apr 2026 · CoreTide#60

Project TideKit

The plan: turn CoreTide into a versioned Python package with a CLI.

16 Jun 2026 · CERT-EU AI workshop

Harness over query

The query is a commodity; harness and intent are the differentiator.

22 Jun 2026 · new repository

opentide, phases 0 to 9 on day one

Executed by an agent fleet, reviewed and merged the same day.

9 Sep → 7 Oct 2026 · PyPI

0.1.0, then 0.11.0

Six repositories archived. 26 releases in four weeks.

Our own issue tracker made the case.

CoreTide#3 2025-04-03

Error when running file_name.py

“Looks like commit 9ef7a58 introduced a new bug.” Client CI pulled the engine at run time.

`upgrades not pinned`

InitTide#5 2025-03-14

Invalid .gitlab-ci.yml

“Includes tide_remote.yml from CoreTide … which in turn includes some more files.”

`fragile CI glue`

InitTide#12 2026-01-01

Sample Detection Rule does not pass…

Expected behaviour, “but it can be (understandably) confusing for newcomers”.

`day-one onboarding`

CoreTide#61 2026-06-22

Phase 0: baseline the engine

“python Orchestration/validate.py fails outside CI.”

`scripts only ran in CI`

CoreTide#65 2026-04-13

Phase 4: Pydantic code-first

A “triple-maintenance data framework”: YAML metaschemas, dataclasses, manual conversion.

`schema drift`

CoreTide#71 2026-04-13

Phase 9: testing suite

“Zero unit tests: validation is entirely pipeline-driven.”

`no tests`

“Consumed as a git submodule with `sys.path.append()` hacks, a 1,546-line monolith entry point.” CoreTide#60

The engine lived inside your repo. That had to change.

before **CoreTide inside your repo** submodule, or cloned at CI time

Orchestration/*.py

```
import sys, git
sys.path.append(str(git.Repo(".",
    search_parent_directories=True).working_dir))
from Engines.modules.tide import DataTide
from Engines.modules.plugins import DeployTide

rule = DataTide.Models.mdr[uuid]
DeployTide.mdr["sentinel"].deploy(...)
```

```
$ git submodule update --init --recursive
$ python Orchestration/validate.py
```

after **opentide as a package** pinned by version

any script, any tool

```
from opentide import Opentide

Opentide.initialise()
rule = Opentide.Rules[uuid]
rule.deploy("sentinel")
```

```
$ pip install 'opentide==0.11.0'
$ opentide validate --strict
$ opentide deploy --platform sentinel --dry-run
```

~~CI pulled the engine at run time. One upstream commit broke pipelines.~~

→ Pin `opentide==0.11.0`

~~Seven Orchestration scripts, no CLI, and they only ran in CI.~~

→ One entry point: `opentide`

~~CI glue through remote includes, broken by GitLab quirks.~~

→ `opentide setup ci` generates it

The plan, written down in April: Project TideKit.

“A versioned Python package on PyPI with a CLI, MCP server, and clean API surface.”

CoreTide#60 · Project TideKit · 13 April 2026

01 CoreTide#60

New repository, not a rewrite in place

Every pinned submodule and clone keeps working while instances migrate.

02 CoreTide#60

A registry, not an orchestrator

The SDK holds the objects. The CLI owns CI context and exit codes.

03 CoreTide#67

One entry point for everything

“Every operation … accessible from a single entry point.”

04 opentide#107

Platforms are repo config

Enabled per repository by setup, not picked at install time.

05 opentide#64 · #28

Onboarding is a command

`setup` replaces the template; CI is generated, not copied.

06 opentide#104

No generic migrate command

Migration is an agent prompt and a diff a human reviews.

Designed “to be executed by an autonomous agentic fleet”: **phases 0 to 9 merged on 22 June, the day the repository was created.**

One tool on your PATH, not scripts in your tree.

Why a CLI and not just a library: ergonomics is where adoption lives, for people and for agents.

ergonomics	submodule era	`opentide` CLI	evidence
Install	Clone, init the submodule, patch sys.path	`pip install opentide`, Python 3.10–3.14	CoreTide#60
Upgrade	CI pulled whatever the engine was today	Pin a version, read the CHANGELOG	CoreTide#3
Onboard	Copy a template; the sample rule failed	`opentide setup`: wizard or scripted flags	InitTide#12 · #64
Operate	Seven scripts that only ran in CI	One command tree, `--help` in 0.2 s, completion	CoreTide#61 · #142
Automate	Remote CI includes, GitLab quirks	`setup ci` for GitHub, GitLab, Azure	InitTide#5 · #28
Read results	Logs, plus a legacy exit code 19	`--json`, exit codes 0 · 1 · 2, file:line	#135 · #139
Run anywhere	Assumed a CI runner	Smoke-tested on Linux, macOS, Windows	#140 · #141
Work with agents	Agents shell out and guess	MCP server and skills, wired by `setup`	CoreTide#68 · #59

issue and PR backlogs: opentidehq/CoreTide, InitTide, opentide (#n = opentide)

Six repositories archived. One coherent stack.

History stays. Pinned submodule SHAs keep resolving. New work goes into the package.

~~CoreTide~~	engine as a submodule	→	**opentide** — CLI · SDK · MCP on PyPI
~~InitTide~~	template repository	→	**opentide setup** — repo, CI, MCP, skills
~~ShareTide~~	shared public objects	→	**library** — 246 threats · 11 objectives · 8 rules
~~WikiTide~~	generated wiki dump	→	**generate docs + explorer** — docs and an object UI
~~AgentTide~~	agent skills	→	**skills** — 27 skills, multi-harness
~~SyntaxTide~~	LSP experiment	→	**language-server** — KQL · SPL · Tide YAML `coming`

An agent is only as good as the spec it is held to.

	what an agent needs	submodule era	`opentide 0.11`
01	**A contract it can read**	JSON Schemas generated in-tree; no separate spec	Normative specs, generated JSON Schema, 36 versioned vocabularies
02	**Feedback it can act on**	Scripts and logs to interpret	`--json` results, exit codes, `file:line` findings
03	**Tools it can call**	None: shell out to scripts	MCP: search, chaining, coverage, validation, dry-run deploy
04	**Knowledge it can load**	One AGENTS.md in a separate repo	27 skills + AGENTS.md via `opentide setup skills`
05	**Gates it cannot skip**	CI glue rewritten per instance	`setup ci`: strict validate and lint, staging first, approved promotion

A submodule gave agents scripts to run. A package gives them a spec to follow.

The CLI is the agent's API.

Every command answers the same way, so pipelines and agents act on results, not prose.

zsh · detection-repo

```
$ opentide --json validate query --platform crowdstrike
{
  "platform": "crowdstrike",
  "valid": null,
  "supported": false,
  "ok": false,
  "status": "failed",
  "message": "query validation not supported for crowdstrike"
}
$ echo $?
1
```

One JSON document on stdout

"ok": true or false, a status, a message. Nothing else.

Diagnostics on stderr

Logs never prefix or corrupt the document.

Exit codes gate CI

0 success · 1 error · 2 usage. No parsing needed.

Explicit by design

Unsupported checks fail with a reason CI can gate on.

One setup wires the repo for humans, CI and agents.

zsh · detection-repo

```
$ pip install 'opentide[mcp]==0.11.0'
$ opentide setup --yes --platform sentinel --ci github
$ opentide setup mcp --claude-code --yes
$ opentide setup skills --yes --claude-code \
    --install opentide-detection-rule
```

ci	platforms
GitHub · GitLab · Azure DevOps	seven adapters, enabled per repo
mcp hosts	**skills targets**
VS Code · Cursor · Claude Code	Claude · Cursor · Copilot · .agents

what you get

```
detection-repo/
├── objects/
│   ├── threats/  objectives/  rules/
├── .opentide/configurations/platforms/
│   └── sentinel.toml
├── .github/workflows/      # validate · lint · deploy
├── .mcp.json               # the opentide MCP server
├── CLAUDE.md               # the agent's entry point
└── .claude/skills/
    └── opentide-detection-rule/
```

Generated pipelines install the exact version you ran setup with: `opentide==0.11.0`. Upgrades arrive as a reviewed diff.

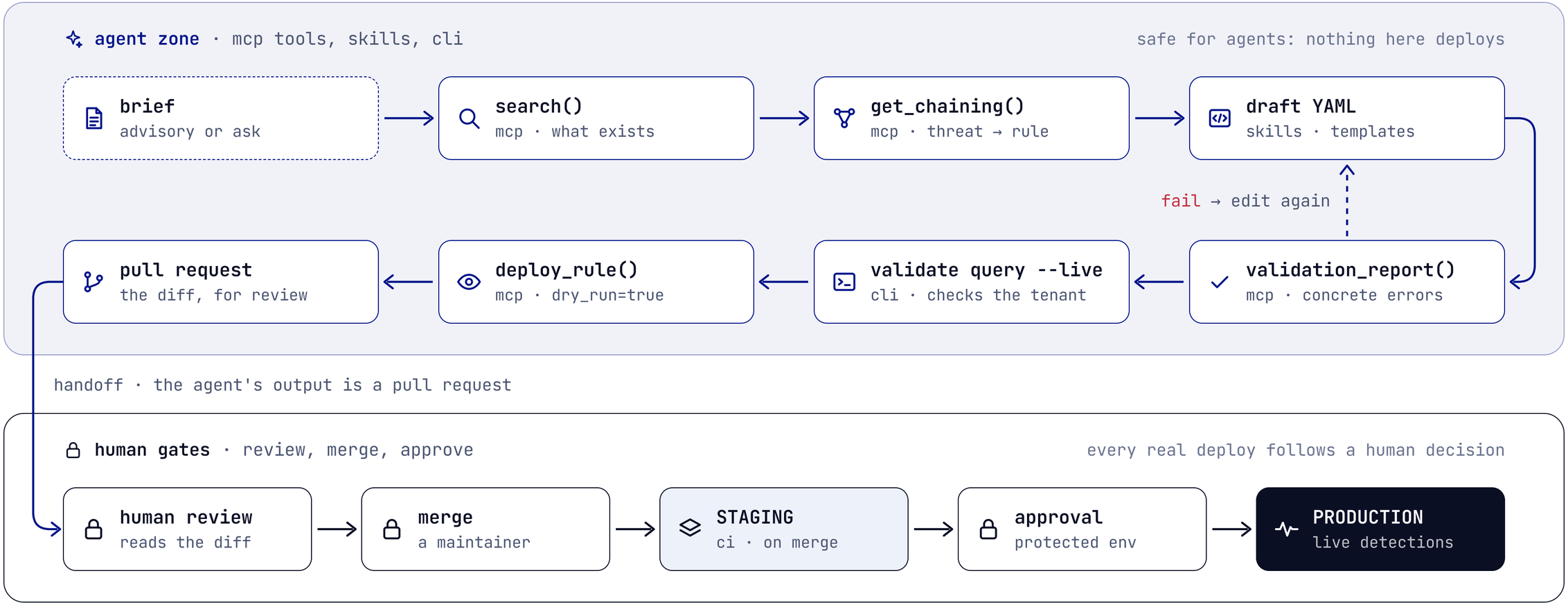
04 / 05

Agentic detection engineering

Agents draft. The engine checks. Humans decide what reaches production.



The loop: agents draft, the engine checks, humans promote.



From a SOC Prime threat report to a linked chain.

prompt

Turn SOC Prime’s Threat of the Month, Gunra ransomware, into Sentinel and Splunk detections. Strict validation, staged deploy.

intel: SOC Prime Active Threats, Threat of the Month

agent session — opentide mcp · skills

```
> open socprime: Gunra → T1190 · T1003
> search(query="T1190") → no threat yet
+ write objects/threats/gunra-ransomware-intrusion.yaml
> search(technique="T1003.001") → objective credential-access exists
> get_chaining(uuid="f8817f95-…") → no rule implements it
~ edit objects/objectives/credential-access.yaml → threats += aa36f6de-…
+ write objects/rules/lsass-memory-access.yaml → sentinel · splunk
> validation_report() → 0 errors
> deploy_rule(uuid="c8f282eb-…", platform="sentinel", dry_run=true) → action: dry-run
```

threat — new

Gunra ransomware intrusion

T1190 · T1003 · Windows::Server

objective — updated

Credential access via LSASS

+1 threat · aa36f6de-…

rule — new

LSASS memory access

rule::1.1 · KQL + SPL · STAGING

Then it meets the same gates as any human change.

!142 merge request

merged into main

Detect LSASS credential dumping in Gunra ransomware intrusions

feat/lsass-credential-access → main

detection-content · sentinel · splunk

Agent-drafted from SOC Prime's Gunra report: a new threat, the LSASS objective linked, a new rule for Sentinel and Splunk.

3 files changed

```
+ objects/threats/gunra-ransomware-intrusion.yaml
~ objects/objectives/credential-access.yaml
+ objects/rules/lsass-memory-access.yaml
```

SOC lead approved these changes ✓

pipeline · from opentide setup ci

Validate — 48s

```
$ opentide validate --strict
$ opentide validate query --platform sentinel --live
$ opentide validate query --platform splunk --live
```

Generate — 33s

```
$ opentide generate   schemas · exports · docs
```

Deploy Staging — 1m 12s

```
$ opentide deploy --plan STAGING
→ Microsoft Sentinel    [STAGING] LSASS memory access
→ Splunk ES             [STAGING] LSASS memory access
```

Deploy Production — not run

Protected environment, separate approval.
This merge only reaches STAGING.

Agents build on what the gates return.

mcp tool	status
search · get_chaining · coverage	supported
validate_rule · validation_report	supported
deploy_rule returns action: dry-run	dry-run by default
validate_query refuses otherwise · then checks the tenant	only with live: true
run_query stub: true · rows: null	not implemented
crowdstrike · harfanglab query checks: deploy-only	supported: false

⚠ found by a first-user simulation · fixed in days

#245 sep 22	**MCP `validate_query` returns `valid=true` for garbage queries** “Agents that trust the tool … will treat invalid KQL/SPL as valid.”
PR #275 sep 22	**Stubs said `rows: 0`. Now: `stub: true`, `rows: null`** rows: 0 is what “an agent reads as ‘the query ran and matched nothing’”
#456 oct 7	**“That offline result is not a tenant check.”** 0.11.0: bare `validate query` refuses and names `--live`

opentide --json validate query --platform sentinel — exit 1

```
{
  "platform": "sentinel",
  "supported": true,
  "ok": false,
  "status": "failed",
  "message": "Query validation checks the tenant. Re-run with --live."
}
```

Strict gates keep agent output threat-aligned and shippable.

What agents may do, and what stays human.

Let agents

- Search, read and summarise objects and coverage
- Draft objects from templates
- Run `validation_report` and surface concrete errors
- Fix dangling references and vocabulary violations
- Preview deploys with `dry_run=true`

Keep a human in the loop

- Real (non-dry-run) deploys to production
- Bulk promotion to `PRODUCTION`
- Editing `.opentide/configurations/` credentials or deployment plans
- Trusting a query check that was not run against the tenant
- Regenerating or hand-editing UUIDs

AGENTS.md — guardrails, from the docs

```
- Always run `validation_report` before proposing a rule as done.
- Never claim query validation for CrowdStrike or HarfangLab — they are deploy-only.
- Deploys are `dry_run=true` unless a human explicitly approves a real deploy.
```

We build opentide the way we want detections built.

163

issues closed since 0.1.0

sep 9 → oct 8 · bugs, docs, setup, migration

150

pull requests merged since 0.1.0

each one merged by a maintainer

133

of them drafted on cloud-agent branches

cursor/… branches

133 / 150

3.10–3.14

Python versions in the CI matrix

tests on each · lint: ruff + ty

85%

coverage gate

CodeQL on every push and PR

20

bugs from a first-user simulation

#239–#258 on 0.3.0 · fixed in 0.4.0 next day

Agents draft. Gates decide. A maintainer merges.

05 / 05

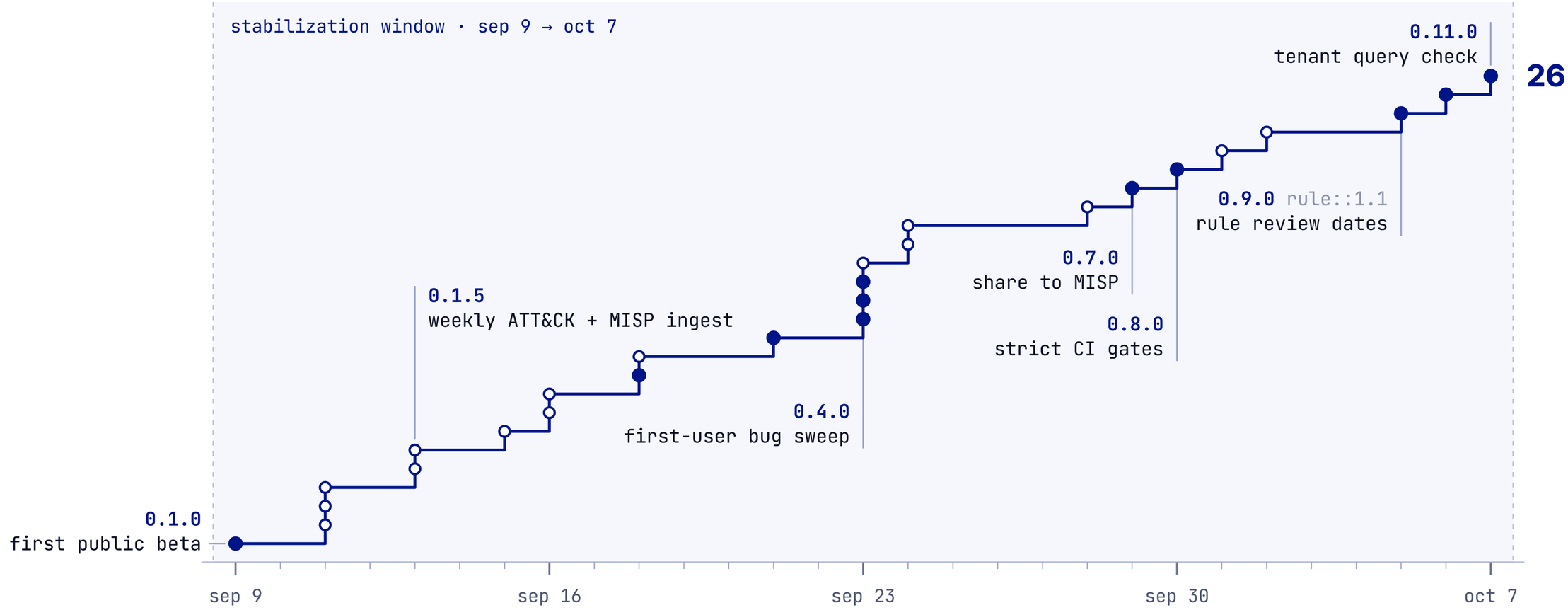
What comes next

Four weeks of stabilization are done. Next: more platforms, editor tooling, and the road to 1.0.

26 releases in four weeks.

● 11 minor ○ 15 patch

The stabilization window: bugs, docs, deployers, agent setup and migration. No new platforms.



Where opentide goes next.

Platforms

Elastic Security

deploy via the detection engine

RFC 0007 · accepted

Sysdig Secure

Falco-native rule schema

RFC 0004 · draft

today

7 platforms

5 with a tenant query check
2 deploy-only

issues #386 · #143

In the editor

Native language server

KQL · SPL · Tide YAML
Rust + tree-sitter
ships inside `opentide[lsp]`

in progress

VS Code extension

replaces `opentide setup vscode`

planned

One-click agent plugins

skills + MCP in one install

planned

issues #332 · #431

A stronger model

`rule::2.0` alongside `rule::1.x`

both shapes in one workspace

planned

Portable rule validation

a disabled platform warns, not fails

planned

Deployment state out of YAML

vendor rule ids in state shards

planned

Schemas for config TOML

Pydantic models + JSON Schema

planned

issues #394 · #410 · #396 · #374

Toward 1.0

CoreTide shims removed

legacy import paths end at 1.0

planned

`--logs json|verbose|debug`

replaces `--json`, adds `--version`

planned

Per-platform skill packs

sentinel-ops, splunk-ops, …

planned

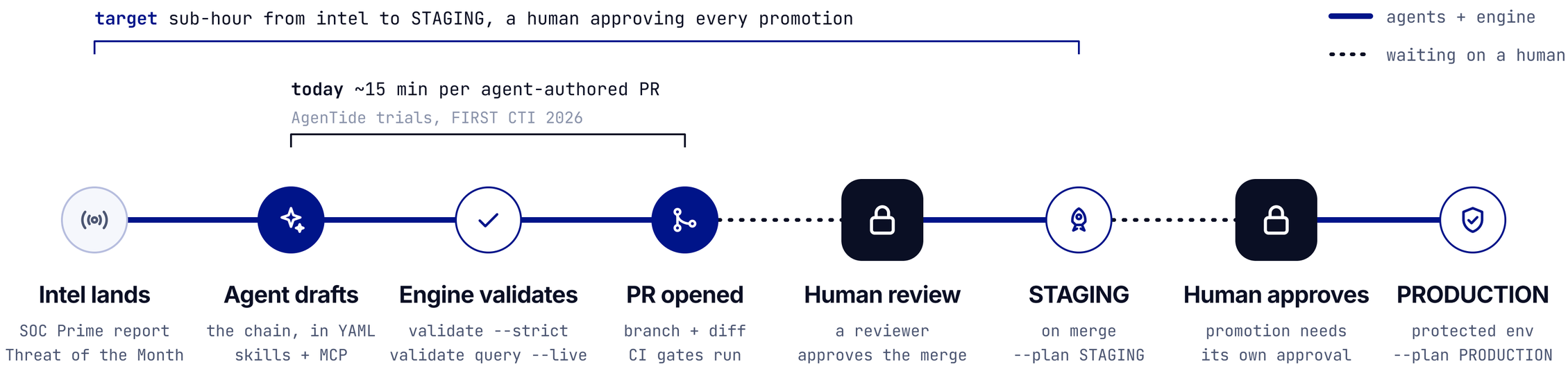
RFC-driven specs

issue → RFC → spec → PR

RFC 0001 · accepted

issues #187 · #50

The north star: intel to reviewed, deployed detection in under an hour.



> **“You choose how much runs with you — and how much runs on its own.”** opentide.org

Start in five commands.

From an empty folder to a validated detection repo, with your agent wired in.

a new detection repo — python 3.10 – 3.14

```
# pin the engine, with the MCP server extra
$ pip install 'opentide[mcp]==0.11.0'

# scaffold a detection repo: Sentinel + GitHub CI
$ opentide setup --yes --platform sentinel --ci github

# wire your agent in: writes .mcp.json
$ opentide setup mcp --claude-code --yes

# schemas, templates, vocabularies, snippets
$ opentide generate

# the same strict gate your CI runs
$ opentide validate --strict
```

home	opentide.org
docs	opentide.org/docs/usage
source	github.com/opentidehq
package	pypi.org/project/opentide
from CoreTide	opentide.org/docs/usage/migration

scan

opentide.org

opentide

Agents write the queries. The spec keeps them aligned.

Ontology, taxonomy, engine and gates. Open source, EUPL-1.2, forever free.

opentide.org github.com/opentidehq pip install opentide